

01

Foundation

Reliable APIs and Core Engineering Practices

DURATION

14 weeks

WEEKLY EFFORT

5-10 hours per week

CAPSTONE

Task Manager API

Purpose and expectations

Build the engineering habits required to design, implement, test, secure, and operate a production-ready backend service. The level combines protocol fundamentals with disciplined application architecture and a portfolio-grade capstone.

RECOMMENDED DURATION**14 weeks****WEEKLY COMMITMENT****5-10 hours per week****ENTRY CRITERIA****Node.js/TypeScript, Git, and programming fundamentals****PRIMARY PROJECT****Task Manager API****CORE FOCUS****HTTP, API design, SQL, testing, security, Docker, and delivery automation****How to use this roadmap**

1. Study for understanding, then validate the idea through implementation.
2. Keep evidence: source code, tests, measurements, diagrams, and short decision records.
3. Prefer a working, explained trade-off over a large but unfinished feature set.
4. Complete each assessment from a clean checkout before advancing.

Capabilities at completion

- Explain the HTTP request-response lifecycle, status codes, headers, TLS, and the practical differences among HTTP/1.1, HTTP/2, and HTTP/3.
- Design predictable REST APIs with resource-oriented naming, versioning, filtering, cursor pagination, and RFC 7807 problem details.
- Model relational data, write joins and window queries, select useful indexes, and use ACID transactions and migrations safely.
- Analyze time and space complexity using Big O notation and implement core search, sorting, recursion, and two-pointer techniques.
- Apply clean code, SOLID, Repository, Factory, and Adapter patterns without over-engineering.
- Write focused unit tests with Jest, test doubles, TDD, and meaningful coverage targets.
- Implement access and refresh token flows, password hashing, authorization middleware, and baseline OWASP controls.
- Package services with multi-stage Docker builds, health checks, networks, and persistent volumes.
- Create a CI/CD pipeline for linting, testing, building, and publishing deployable artifacts.
- Produce structured logs and shut down gracefully by draining connections and releasing resources.

Roadmap at a glance

PHASE	SCHEDULE	FOCUS	EVIDENCE
Core web	Weeks 1-2	HTTP, TLS, API contracts	API design brief and OpenAPI contract
Data and algorithms	Weeks 3-5	SQL, query performance, Big O	Database model and algorithm notes
Code quality	Weeks 6-8	SOLID, testing, authentication	Tested authentication service
Operations	Weeks 9-10	Docker, CI/CD, logging	Containerized build and pipeline
Capstone	Weeks 11-14	Integration, review, assessment	Production-ready Task Manager API

Completion rule

Do not treat elapsed time as completion. Advance when you can demonstrate the deliverables, explain the design decisions, and reproduce the result.

Weekly learning plan

WEEK 01

HTTP and HTTPS: How the Web Works

5h study + 4h practice

Develop a protocol-level mental model before building abstractions on top of it.

LEARNING FOCUS

- Request-response lifecycle over TCP; safe and idempotent method semantics.
- Status code families and common API responses, including 201, 204, 401, 403, 409, 422, and 429.
- Request, response, caching, CORS, and security headers.
- TLS 1.3 handshake, certificate chains, confidentiality, integrity, and authentication.
- HTTP/2 multiplexing and header compression; HTTP/3 over QUIC.

APPLIED WORK

- Inspect requests and response headers with `curl -v` and `curl -I`.
- Inspect a TLS 1.3 handshake with `openssl s_client`.
- Validate conditional requests with ETag and If-None-Match.
- Document which methods are safe, idempotent, and expected to carry a body.

DELIVERABLE Create `http-foundations.md` with method and status-code matrices, a TLS handshake diagram, and an HTTP/1.1 vs HTTP/2 vs HTTP/3 comparison.

WEEK 02

Professional API Design

4h study + 4h practice

LEARNING FOCUS

- Resource-oriented, plural endpoint naming and consistent URL conventions.
- Versioning through paths, headers, or media types; select one strategy deliberately.
- Offset and cursor pagination, filtering, sorting, and sparse fieldsets.
- Consistent error responses based on RFC 7807 Problem Details.
- Contract-first delivery with OpenAPI before implementation.

APPLIED WORK

- Design the complete Task Manager API surface.
- Specify success and failure responses for every operation.
- Define cursor semantics and stable ordering.
- Review the contract against Microsoft REST API Guidelines and Google API design guidance.

RFC 7807 EXAMPLE

```
{
  "type": "https://api.example.com/problems/validation",
  "title": "Validation failed",
  "status": 422,
  "detail": "One or more fields are invalid.",
  "errors": [{ "field": "email", "message": "Enter a valid email address." }]
}
```

DELIVERABLE A reviewed OpenAPI contract that can drive implementation and tests.

WEEK 03 Relational Modeling and SQL

3h study + 5h practice

LEARNING FOCUS

- Normalization through 3NF, primary and foreign keys, constraints, and indexes.
- Joins, grouping, subqueries, and transaction boundaries.
- Reading query plans with EXPLAIN ANALYZE.

APPLIED WORK

- Model users, tasks, and categories.
- Write migrations and CRUD queries.
- Create evidence-based indexes and exercise BEGIN, COMMIT, and ROLLBACK.

DELIVERABLE Versioned SQL migrations, seed data, representative queries, and a short index rationale.

WEEK 04 Advanced SQL: Windows, CTEs, and Performance

3h study + 4h practice

LEARNING FOCUS

- ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, and NTILE.
- Recursive and non-recursive common table expressions.
- Sequential, index, and bitmap scans in query plans.

APPLIED WORK

- Rank users by completed tasks.
- Compare equivalent query shapes.
- Record plan changes before and after indexing.

WINDOW QUERY

```
SELECT
  u.name,
  COUNT(t.id) AS tasks_done,
  ROW_NUMBER() OVER (ORDER BY COUNT(t.id) DESC) AS rank,
  LAG(COUNT(t.id)) OVER (ORDER BY COUNT(t.id) DESC) AS previous_count
FROM users u
JOIN tasks t ON t.user_id = u.id AND t.status = 'done'
GROUP BY u.id;
```

DELIVERABLE A query-performance note containing plans, observations, and the final optimized query.

WEEK 05 Big O and Essential Algorithms

4h study + 4h practice

LEARNING FOCUS

- Complexity of arrays, linked lists, hash maps, and trees.
- Binary search, quicksort, merge sort, recursion, memoization, sliding windows, and two pointers.

APPLIED WORK

- Implement and analyze binary search, sorting, palindrome, Fibonacci, and two-sum solutions.
- Solve five introductory interview problems.

- How input constraints influence algorithm selection.

- Explain both time and space complexity for each solution.

DELIVERABLE An algorithms notebook with five solved problems, complexity analysis, and trade-offs.

WEEK 06

Clean Code, SOLID, and Error Handling

4h study + 5h practice

LEARNING FOCUS

- Clear naming, cohesive functions, consistent formatting, DRY, and KISS.
- SRP, OCP, LSP, ISP, and DIP in application code.
- Layered responsibilities across controllers, services, and repositories.
- Domain-aware errors mapped to stable API responses.

APPLIED WORK

- Refactor a deliberately tangled implementation.
- Introduce repository and logger interfaces.
- Document the before-and-after dependency direction.

DEPENDENCY INVERSION

```
class TaskService {
  constructor(
    private readonly repository: TaskRepository,
    private readonly logger: Logger,
    private readonly notifications: NotificationService,
  ) {}
}
```

DELIVERABLE A small refactoring case study demonstrating SRP and DIP with tests.

WEEK 07

Unit Testing and TDD

3h study + 5h practice

LEARNING FOCUS

- Test pyramid and the boundary between unit, integration, and end-to-end tests.
- Mocks, stubs, spies, and semantic test doubles.
- Red-green-refactor without coupling tests to implementation details.

APPLIED WORK

- Test task validation with a TDD loop.
- Test authentication success and failure paths.
- Measure coverage while prioritizing business behavior over percentages.

DELIVERABLE Authentication service tests for registration, login, refresh, expiration, and invalid credentials, with at least 80% meaningful coverage.

LEARNING FOCUS

- Access vs refresh token responsibilities and lifetimes.
- Password hashing with bcrypt and appropriate work factors.
- Refresh-token rotation, revocation, secure storage, XSS, and CSRF considerations.
- Authentication and role-based authorization middleware.

APPLIED WORK

- Implement register, login, refresh, and logout endpoints.
- Keep access-token claims minimal.
- Persist and revoke refresh-token families.
- Test expired, replayed, and revoked tokens.

DELIVERABLE A documented authentication flow with threat notes and automated tests.

LEARNING FOCUS

- Layers, build cache, multi-stage images, and .dockerignore.
- Non-root runtime users, health checks, restart policies, volumes, and networks.
- Dependency readiness and environment configuration.

APPLIED WORK

- Build a minimal production image.
- Run the API and PostgreSQL with Compose.
- Verify health checks, persistent data, and network isolation.

PRODUCTION IMAGE PATTERN

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine AS runtime
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/package*.json ./
RUN npm ci --omit=dev && npm cache clean --force
USER node
EXPOSE 3000
CMD ["node", "dist/index.js"]
```

DELIVERABLE A reproducible local environment and a small, non-root production image.

LEARNING FOCUS

- GitHub Actions triggers, jobs, services, artifacts, caches, and secrets.
- Machine-readable logs with timestamp, level, message, request ID, user ID, and service name.
- Signal handling, connection draining, and ordered resource cleanup.

APPLIED WORK

- Create lint, test, build, and image-publish stages.
- Add PostgreSQL as a pipeline service.
- Use structured logs in request flows.
- Handle SIGTERM without dropping in-flight requests.

DELIVERABLE A green CI pipeline, correlated JSON logs, and a verified graceful-shutdown sequence.

WEEKS 11-12

Capstone: Task Manager API

Integrate the entire level into one coherent, portfolio-ready service.

LEARNING FOCUS

- Clean Architecture boundaries and dependency injection.
- Cursor pagination, filters, validation, and RFC 7807 errors.
- Authentication, structured logging, graceful shutdown, testing, and API documentation.

APPLIED WORK

- Build with Node.js/TypeScript, PostgreSQL, Docker, Jest, Zod, Pino, and OpenAPI.
- Organize domain, application, infrastructure, and main layers.
- Publish a CI-tested container and document local startup.

ARCHITECTURE AND SCOPE

- Domain: entities and pure business rules
- Application: use cases and ports
- Infrastructure: PostgreSQL, JWT, HTTP, and logging adapters
- Main: dependency wiring, server lifecycle, and configuration

DELIVERABLE A public repository with a working API, automated tests, OpenAPI documentation, Docker Compose, and a concise architecture decision record.

WEEK 13

Guided Review and Refactoring

LEARNING FOCUS

- Self-review for oversized functions, poor names, hidden coupling, and SRP violations.
- Query review with EXPLAIN ANALYZE.
- Consistency across errors, logs, and repository abstractions.

APPLIED WORK

- Run a structured code review against a checklist.
- Refactor the highest-risk areas only.
- Capture before-and-after evidence in the pull request.

DELIVERABLE A focused refactoring pull request with measured improvements and no feature expansion.

WEEK 14

Level Assessment

LEARNING FOCUS

- Explain 401 vs 403, 200 vs 201, and the TLS handshake.
- Defend cursor pagination and write a CTE with a window function.
- Identify SOLID violations, explain multi-stage builds, and compare lookup complexity.
- Explain refresh-token security and common browser threats.

APPLIED WORK

- Answer at least six of eight technical questions accurately.
- Demonstrate the Task Manager API from a clean checkout.
- Walk through one design trade-off and one production failure scenario.

DELIVERABLE Pass when the technical threshold is met and the complete capstone is available in GitHub with reproducible setup instructions.

Recommended source material

Protocols and API design

RFC 9110; RFC 8446; RFC 7807; Microsoft REST API Guidelines; Google API Design Guide.

Data and architecture

PostgreSQL documentation; Clean Code; Refactoring Guru; Martin Fowler on test doubles.

Security and operations

OWASP guidance; JWT documentation; Docker best practices; GitHub Actions documentation.

Algorithms

Grokking Algorithms plus problem practice with written complexity analysis.

Completion perspective

Level 1 is complete when you can build and explain a reliable backend service end to end, not merely when the checklist is filled in.